ATML PA3: Colab A100 command guide

This is an execution guide, not the graded report. Full experiments have not been run in this delivery. Run them to produce real evidence, inspect the results, complete annotations, and write your own report.

1. Select hardware and load the source

In Colab choose Runtime > Change runtime type > A100 GPU. An A100 40 GB is the target; availability depends on your Colab plan. Keep a persistent copy of this source and all external inputs on Drive.

Clone the source repository using this cell:

```
!git clone https://github.com/Bulibuh/28100053-ATML-PA3.git /content/atml-pa3
%cd /content/atml-pa3
```

Alternatively, upload the delivered source ZIP from your computer:

```
from google.colab import files
import zipfile
uploaded = files.upload()  # select atml-pa3-deliverable.zip
bundle = next(name for name in uploaded if name.endswith('.zip'))
with zipfile.ZipFile(bundle) as archive:
    archive.extractall('/content')
%cd /content/atml-pa3
```

Use one source-loading method, not both. A new Colab runtime loses /content; Drive copies survive.

2. Install and verify

The installer supports Python 3.11–3.13, including Colab's Python 3.13 runtime. Run this cell from the repository directory:

```
%%bash
set -euo pipefail
python scripts/setup_colab.py
```

It creates .venv-colab without shared site-packages, installs the official matching PyTorch 2.9.1 / torchvision 0.24.1 CUDA 12.8 wheels, installs assignment dependencies, runs pip check, checks compiled torchvision operators and model imports, executes an FP16 CUDA matrix multiplication, and runs the tests. Colab's notebook-kernel packages are left untouched. The first install downloads a separate CUDA stack and needs several GB of disk; subsequent runs reuse it.

Inside a %%bash cell, use ordinary shell commands: python, never !python. Do not put a backslash at the end of every line. A backslash is used only for the deliberate multiline commands shown later. Exit code 127 means a shell command was not found; IPython ! syntax is not recognized inside bash.

Activate the environment in EVERY later %%bash cell. Activation does not carry over between cells:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
python -m pip check
pa3 doctor --require-cuda
```

The notebook uses this same isolated interpreter for all subprocesses. Do not import the notebook kernel's preinstalled torch to judge the assignment environment. If you already ran the old shared-package install or torchvision workaround, use the new isolated installer; no manual torchvision uninstall or --no-deps replacement is needed. The --cpu setup option is for implementation checks on a CPU machine and must not be used for A100 experiments. CPU test success does not establish GPU readiness.

3. Mount Drive and checkpoint safety

```
from google.colab import drive
from pathlib import Path
import os

drive.mount('/content/drive')
backup = Path('/content/drive/MyDrive/ATML_PA3')
backup.mkdir(parents=True, exist_ok=True)
os.environ['HF_HOME'] = '/content/hf-cache'
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
```

Prefer local /content disk for active training; periodically copy recovery states and evidence to Drive. Saving a checkpoint locally does not protect it from a lost runtime. Check free disk before Full FT: each FP32 model plus AdamW recovery file can occupy several GB.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
df -h /content
mkdir -p /content/drive/MyDrive/ATML_PA3/checkpoints
cp -a checkpoints/. /content/drive/MyDrive/ATML_PA3/checkpoints/
cp -a evidence /content/drive/MyDrive/ATML_PA3/
cp -a data/prepared /content/drive/MyDrive/ATML_PA3/
```

Run that copy cell only after the relevant local directories exist. For protection during a long GPU run, use a separate periodic backup process or run one method per session and copy immediately afterward. Do not delete the last good backup when replacing it.

4. Task 1: prepare data and shared baseline

The data command resolves an immutable starting-model revision, filters source-order MetaMathQA against GSM8K test questions, selects 40,000 untruncated examples, and saves fixed probe IDs.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task1 prepare
pa3 task1 evaluate --condition baseline
```

Keep data/prepared/model.json and data/prepared/task1 for all five variants and all later sessions. Do not regenerate a different selection between methods. Baseline evaluation is shared and should run once.

5. Task 1: train all five variants and evaluate

Each method starts independently from Qwen3-0.6B. This cell can run for a long time; running each method in a separate cell/session is equivalent if you preserve prepared data and checkpoints.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task1 train --method full
pa3 task1 evaluate --condition full_mid
pa3 task1 evaluate --condition full

pa3 task1 train --method l2sp
pa3 task1 evaluate --condition l2sp

pa3 task1 train --method lora4
pa3 task1 evaluate --condition lora4

pa3 task1 train --method lora16
pa3 task1 evaluate --condition lora16

pa3 task1 train --method oplora16
pa3 task1 evaluate --condition oplora16
```

Defaults are the prescribed microbatch 2, accumulation 16, maximum 2,048 tokens, 1,250 updates, FP16 autocast/dynamic scaling, FP32 masters/adapters/states, and gradient checkpointing. Full/L2-SP peak LR is 2e-5; adapter peak LR is 2e-4. The implementation retains the L2-SP reference on the A100 to avoid per-step CPU transfers. OPLoRA exact FP32 SVD is cached by model revision. SDPA avoids a custom attention installation.

Recovery files are at updates 312, 625, 937, and 1250. Resume a method from its own checkpoint, never another method's state:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task1 train --method full \
  --resume checkpoints/task1/full/update_625.pt
```

Restore data/prepared and matching checkpoint files from Drive before resuming in a new runtime. AdamW, schedule, scaler, RNG, and data position are recovered. Review overflow.jsonl and overflow_retries in training logs. Overflowed steps are retried on the same batch/RNG, without advancing the data or schedule, so successful-update budgets remain exact. Persistent nonfinite losses stop the run.

6. Task 1: diagnostics and qualitative output

The PDF links the exact upstream YAML and MathVista question metadata. Fetch the inspected versions using the checksum-verified helper. The qualitative reader supports the native YAML directly and preserves JSON system messages; no manual flattening or generated replacement prompts are needed.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
python scripts/download_public_inputs.py

%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task1 diagnostics
pa3 task1 qualitative --prompts /content/atml-pa3/data/external/first_plot_questions.yaml
pa3 evidence
```

The qualitative output contains 168 responses. Complete the 42-row annotations_to_complete.csv yourself for the first and last underlying questions across all formats/conditions. Include excerpts and flags. Diagnostics use layers 0, 13, and 27 and the saved fixed math/non-math probes.

The combined one-command runner is available when the prompt file exists:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
export QUALITATIVE_PROMPTS=/content/atml-pa3/data/external/first_plot_questions.yaml
bash scripts/run_task1.sh
```

Do not run that full runner again after completing the individual commands. It is an alternative orchestration method.

7. Task 2: official annotations and selected image features

Download official annotations. If a published URL has moved, download the same files from the COCO/VQA official download pages and use the folder shown in data/external/README.md.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
python scripts/download_annotations.py
pa3 task2 prepare --annotations data/external/coco/annotations
pa3 task2 cache --download
```

Only selected COCO images are downloaded; the complete multi-GB image archives are unnecessary. The cache command uses frozen SigLIP, applies final layer normalization once through the model's last_hidden_state output, and stores FP16 196x768 features. Qwen starts in a separate process after SigLIP leaves GPU memory. Keep the feature cache to avoid repeating work.

If images are already available, supply their root instead of downloading:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task2 cache --images /content/coco/images
```

For automatic qualitative image sheets, use the default data/external/coco/images folder or a symlink at that location. The prepare command saves image-disjoint development/final-test sets, all references, pair IDs, and fixed mismatched-image mappings. Inspect annotation-derived labels before looking at predictions.

8. Task 2: curriculum and four image controls

Task 2 shares the immutable starting Qwen specification from Task 1 prepare. If you are running Task 2 first, run pa3 task1 prepare once to create it.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task2 train --visual-tokens 196
pa3 task2 evaluate --visual-tokens 196
pa3 evidence
```

The main recipe runs 2,250 caption updates, 1,500 paired grounding/replay updates, and 750 natural-VQA/replay updates, with effective batch 16. Connector FP32 masters are trained throughout; new rank-16 Qwen attention adapters are enabled in Stage 2 and continue through Stage 3. Development likelihood is measured before training, every 500 updates, and at stage endpoints. No final-test result selects a checkpoint.

Evaluation uses batch 8, greedy 16-new-token generation, and correct, complementary-swap, mismatched, and no-image conditions. References remain the original answers. Pair-level bootstrap uses 4,000 resamples.

Resume the same 196-token run from a saved global update, for example:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task2 train --visual-tokens 196 \
  --resume checkpoints/task2/tokens_196/update_2500.pt
```

The optional 49-token experiment requires a fresh independent curriculum. It reuses the frozen 196-patch cache and averages 2x2 spatial blocks before a newly initialized connector:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task2 train --visual-tokens 49
pa3 task2 evaluate --visual-tokens 49
pa3 evidence
```

9. Task 3: place the actual supplied files

The assignment links this exact course Drive folder. In Colab, attempt the following download, or use your authorized Drive/browser access if anonymous download is unavailable:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
python -m pip install gdown==5.2.0
gdown --folder \
  'https://drive.google.com/drive/folders/1utX-Gfi9vvaoi1mi0pzNpDGL1o1fAh23' \
  -O /content/ATML-PA3-Data
```

The cloud machine used for code validation cannot reach this Drive host under its current outbound policy, so the course file schemas and model revision are not claimed as verified. Read the downloaded original README.MD. Locate the LLM and VLM folders, each containing generations.parquet and probability_metrics.parquet; the VLM folder additionally contains attention_metrics.parquet.

Run `python scripts/download_public_inputs.py` to fetch the exact linked MathVista testmini metadata JSON into data/external/. Inspect actual columns before mapping them; the reader will not silently guess the schema:

```
from pathlib import Path
import pandas as pd
root = Path('/content/ATML-PA3-Data')
for path in root.rglob('*.parquet'):
    print(path, pd.read_parquet(path).columns.tolist())
```

Edit configs/rollout_schema.json so its right-hand values match the supplied column names. Keep the left-hand canonical names unchanged. Use group attention totals, aligned raw-distribution KL/log-probability lists, and retained token counts excluding termination. Join original problem text or metadata from released files if necessary; never fabricate it.

10. Task 3: offline analyses and adaptive controllers

Replace the example paths below with the actual archive structure. Missing README/metadata or invalid joins/lengths are errors to resolve, not reasons to omit questions.

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 task3 offline \
  --math-dir /content/ATML-PA3-Data/LLM \
  --mathvista-dir /content/ATML-PA3-Data/VLM \
  --metadata /content/atml-pa3/data/external/mathvista_testmini.json \
  --readme /content/ATML-PA3-Data/README.MD \
  --schema configs/rollout_schema.json
pa3 evidence
```

This is primarily a CPU analysis; it does not need model loading or original MathVista images. It saves 200/800 development/evaluation IDs, ten nested rollout orders, oracle/selection curves, costs, signal diagnostics, candidate-relative VisConf calibration, the tokenwise fusion ablation, development-selected stopping thresholds, held-out results, stopping distributions, and paired question-level bootstrap intervals. The first 50 evaluation MATH questions are saved for the live experiment.

11. Task 3: live A100 generation

Read the exact Qwen2.5-3B-Instruct commit SHA from the supplied README. Do not use an arbitrary current main revision. Replace the placeholder below before running:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
REVISION_FROM_README=REPLACE_WITH_EXACT_40_CHARACTER_MODEL_SHA
pa3 task3 live \
  --revision "$REVISION_FROM_README" \
  --readme /content/ATML-PA3-Data/README.MD
pa3 evidence
```

The script checks the revision against the README, loads FP16 weights without quantization, and uses batch size 1. For each of the fixed 50 MATH questions it generates one 3,072-token response, three independently seeded 1,024-token responses, and one 2,048-token critique/revision using the shared first parallel response. Total response allowances are 3,072 tokens per strategy. Actual tokens, full prompts, seeds, truncation, extraction, runtime, memory, and wrong/correct transitions are saved. The shared initial call is charged to both parallel and refinement costs. No gold answer is sent in prompts or used for selection.

12. Build and inspect evidence

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
pa3 evidence
pa3 audit
python -m pytest -q --junitxml=evidence/validation/pytest.xml
```

Review evidence/status.json, raw configs and logs, and the artifact map in evidence/README.md. Empty or missing experiment files remain incomplete. Check all seven Task 1 conditions and all expected dataset sizes; check Task 2 pair counts and image controls; check both offline datasets and all 50 live questions. The optional 49-token run is not part of the required evidence audit.

The evidence generator builds figures/tables only from existing measured files. It does not invent missing results or write your scientific interpretations. Complete human annotations, choose supported excerpts, and write your own report.

13. Archive and download outputs

Back up large checkpoints separately. Export evidence/configuration/provenance and prepared split IDs without raw datasets or weights:

```
%%bash
set -euo pipefail
source .venv-colab/bin/activate
mkdir -p /content/drive/MyDrive/ATML_PA3
python -m pip list --format=freeze > evidence/colab_environment.txt
tar -czf /content/drive/MyDrive/ATML_PA3/evidence.tar.gz \
  evidence configs docs data/prepared/model.json \
  data/prepared/task1/probe_ids.json \
  data/prepared/task1/selection.json
```

The environment-export command records the actual Colab library versions before archiving. The archive includes only evidence already present; Task 2 and Task 3 selected IDs are recorded within their evidence folders. Missing Task 1 prepared files will stop the archive command and should be resolved or intentionally omitted when archiving a partial run.

Download an evidence archive from Colab if desired:

```
from google.colab import files
files.download('/content/drive/MyDrive/ATML_PA3/evidence.tar.gz')
```

Do not submit command documentation as the 8-page report. Write that report yourself using your actual measured results, cite libraries/datasets/papers, disclose incomplete runs or deviations, and include your accessible public GitHub link as required.